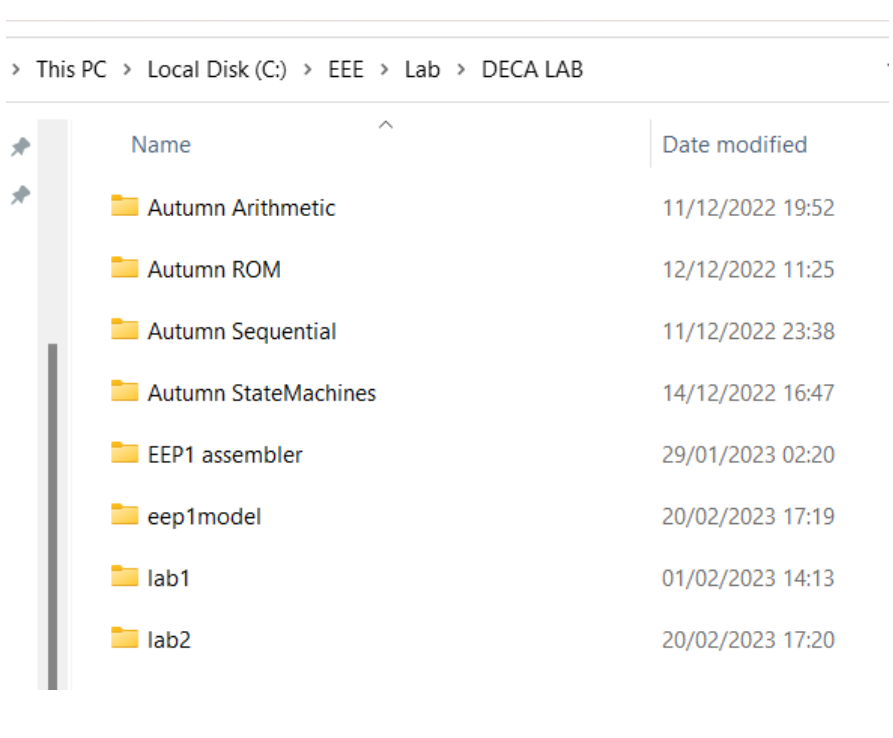
DECA 3.2 - Control Path and Jump Instructions

17 February 2023   15:36

The first step in this section of the lab was to download the new version of the eep1model and lab2 Issie projects from the GitHub. I did so, and placed them in the same directory as the assembler.



› This PC › Local Disk (C:) › EEE › Lab › DECA LAB

Name	Date modified
Autumn Arithmetic	11/12/2022 19:52
Autumn ROM	12/12/2022 11:25
Autumn Sequential	11/12/2022 23:38
Autumn StateMachines	14/12/2022 16:47
EEP1 assembler	29/01/2023 02:20
eep1model	20/02/2023 17:19
lab1	01/02/2023 14:13
lab2	20/02/2023 17:20

I also downloaded the updated version of the EEP1 Assembler before starting this lab, for good measure.

Task 0 - *Work out an algebraic truth table for PCNEXT by taking inputs COND.RET, COND.JUMP, and NEXT.JMP.*

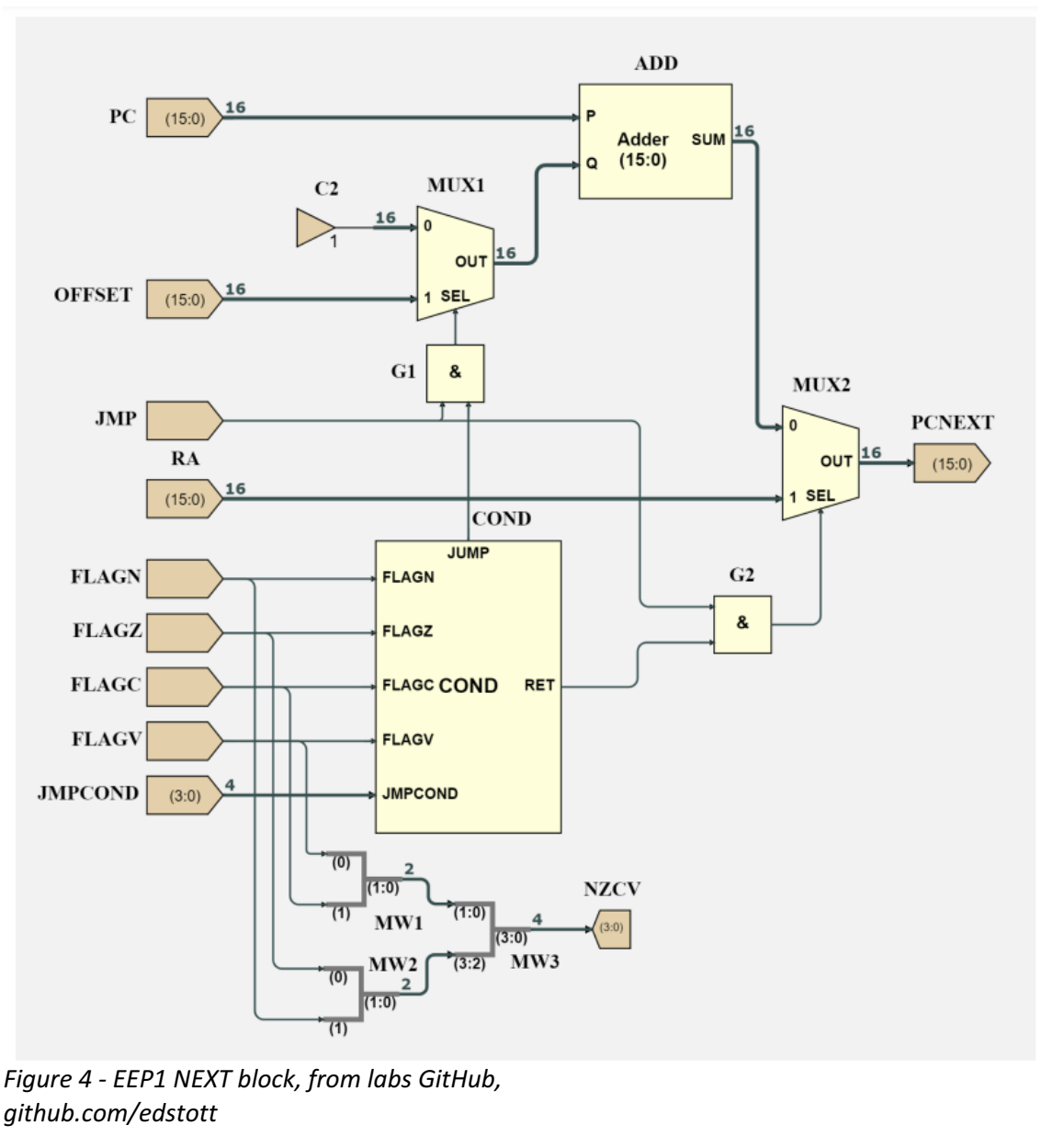


Figure 4 - EEP1 NEXT block, from labs GitHub, github.com/edstott

In the NEXT block, as seen above, the JMP input indicates whether the current instruction is a type of jump or not. If it is, and if the corresponding conditions are met, then the jump will take place. To determine whether the corresponding conditions for the JMP instruction have been met, the COND block takes in the state of all the Flags, and the JMPCOND, which tells it which condition needs to be met for the current instruction. If these conditions are met, the COND block will set output COND.JUMP to 1.

If the current instruction is of type JMP, and if its conditions have been met, then both NEXT.JUMP and COND.JUMP will be equal to 1 --> then the AND gate G2 will also output 1. This output goes to the select line of MUX2, which subsequently passes through the value of RA to the PCNEXT output. Our jump has thus been successfully executed.

The RET output of the COND block does not have any conditions which need to be met: it is an unconditional jump. It is used to return from subroutines. It also does not use Joffset.

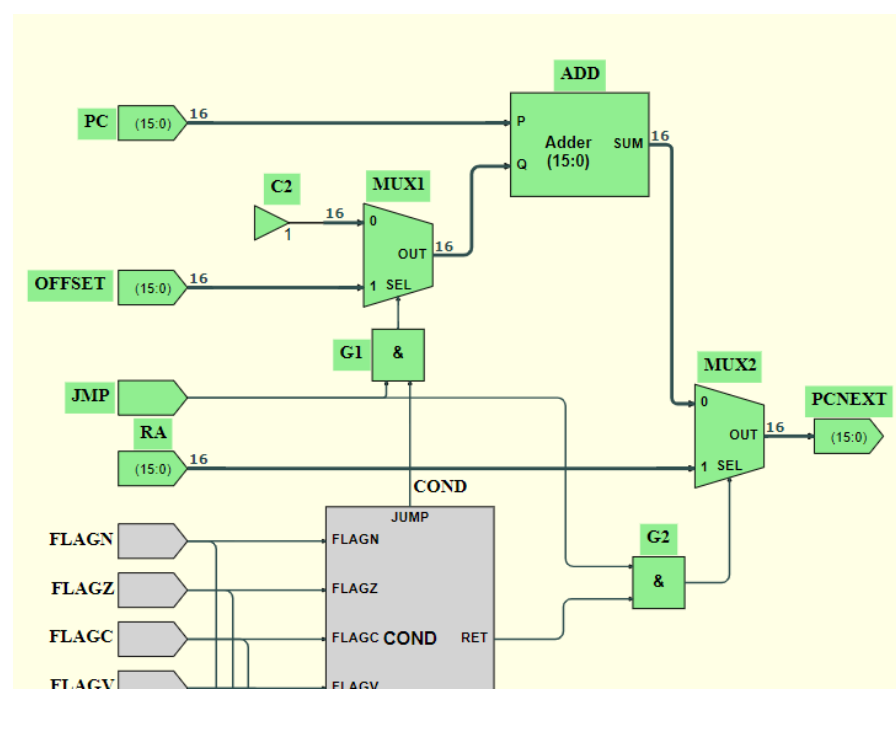
NEXT.JMP	COND.JMP	COND.RET	PCNEXT
0	0	0	PC := PC + 1
0	0	1	PC := PC + 1
0	1	0	PC := PC + 1
0	1	1	PC := PC + 1
1	0	0	PC := PC + 1
1	0	1	PC := RA
1	1	0	PC := PC + OFFSET
1	1	1	PC := RA

By simplifying for don't cares:

NEXT.JMP	COND.JMP	COND.RET	PCNEXT
0	X	X	PC := PC + 1
1	0	0	PC := PC + 1
1	X	1	PC := RA
1	1	0	PC := PC + OFFSET

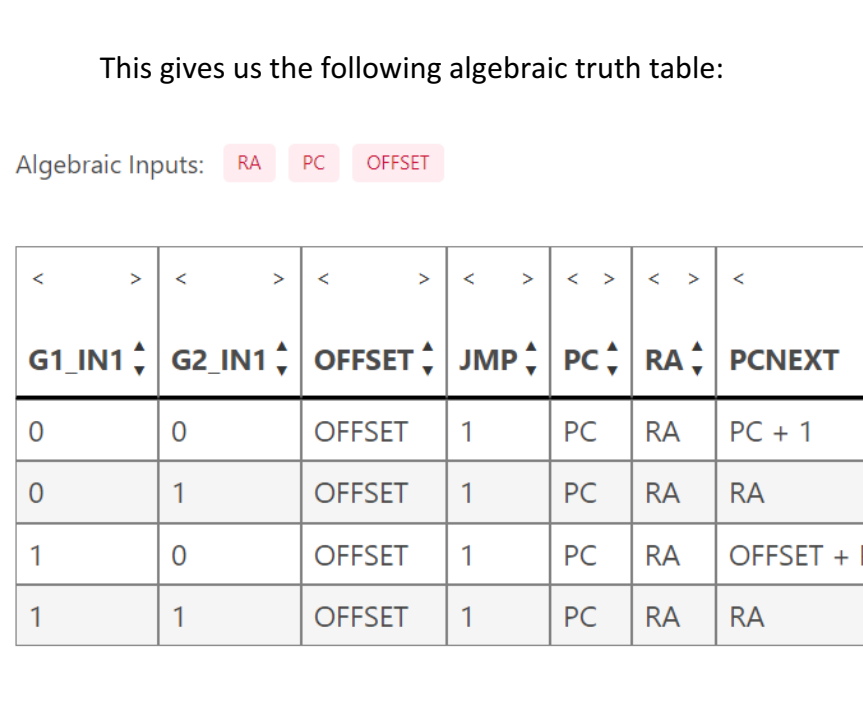
Task 1 - *Create an 8-row Issie algebraic truth table, selecting components as shown in Figure 5, to examine how the NEXT sheet drives NEXT.PCNEXT. Compare the truth table with your hand-generated truth table - they should be the same. You may assume that NEXT.JMP is 1 only for the jump instructions. What is PCNEXT when NEXT.JMP = 0? Compare the truth-table with the EEP1 jump instructions (Figure 2) to determine how inputs COND.JUMP and COND.RET affect the control path operation.*

In the next sheet in Issie, we made an algebraic truth table in order to check the one we had made previously. We selected only the components highlighted in green to be part of the truth table.



We can then impose some input constraints in order to make the truth table more readable. For instance, we can set JMP = 1, since we are not interested in the cases which are not Jump instructions. Additionally, we can set RA, PC, and OFFSET to algebraic inputs, since we do not need to know their numeric value.

This gives us the following algebraic truth table:



Algebraic Inputs: RA PC OFFSET

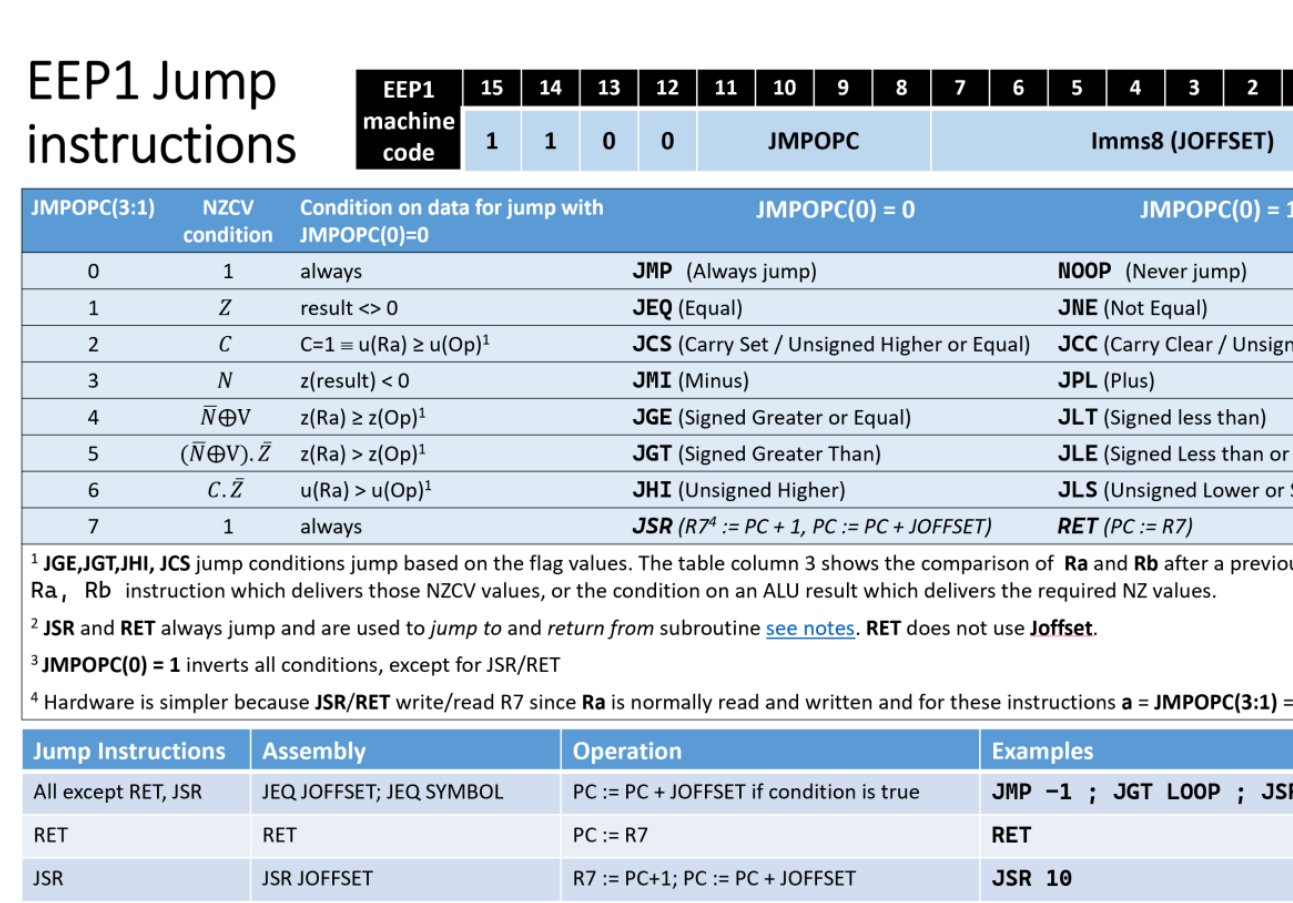
G1_IN1	G2_IN1	OFFSET	JMP	PC	RA	PCNEXT
0	0	OFFSET	1	PC	RA	PC + 1
0	1	OFFSET	1	PC	RA	RA
1	0	OFFSET	1	PC	RA	OFFSET + PC
1	1	OFFSET	1	PC	RA	RA

Note that G1_IN1 is the AND gate G1, which corresponds to COND.JUMP (since JMP is always 1 in this instance). And G2_IN1 corresponds to COND.RET for the same reason.

We can therefore conclude that our previous truth table was accurate.

Task 2 - *In the cond sheet replace the existing constant '1' connection at MUX1 inputs by appropriate logic to implement the jump conditions for JMPOPC = 4,5,6. Show that your logic is correct by using an Issie algebraic truth-table.*

We need to implement combinational logic which implements the conditions for the jump operations JMPOPC = 4, 5, 6.



EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							

JMPOPC(3:1)	NZCV condition	Condition on data for jump with JMPOPC(0)=0	JMPOPC(0) = 0	JMPOPC(0) = 1[3]
0	1	always	**JMP** (Always jump)	**NOOP** (Never jump)
1	Z	result <> 0	**JEQ** (Equal)	**JNE** (Not Equal)
2	C	C=1 = u(Ra) ≥ u(Op)[1]	**JCS** (Carry Set / Unsigned Higher or Equal)	**JCC** (Carry Clear / Unsigned Lower)
3	N	z(result) < 0	**JMI** (Minus)	**JPL** (Plus)
4	$\bar{N}\oplus V$	z(Ra) ≥ z(Op)[1]	**JGE** (Signed Greater or Equal)	**JLT** (Signed less than)
5	$(\bar{N}\oplus V).\bar{Z}$	z(Ra) > z(Op)[1]	**JGT** (Signed Greater Than)	**JLE** (Signed Less than or Equal)
6	$C.\bar{Z}$	u(Ra) > u(Op)[1]	**JHI** (Unsigned Higher)	**JLS** (Unsigned Lower or Same)
7	1	always	***JSR*** *(R7[4] := PC + 1, PC := PC + JOFFSET)*	***RET*** *(PC := R7)*

[1] **JGE,JGT,JHI, JCS** jump conditions jump based on the flag values. The table column 3 shows the comparison of **Ra** and **Rb** after a previous SUB `Ra, Rb` instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.

[2] **JSR** and **RET** always jump and are used to *jump to* and *return from* subroutine see notes. **RET** does not use **Joffset**.

[3] **JMPOPC(0) = 1** inverts all conditions, except for JSR/RET

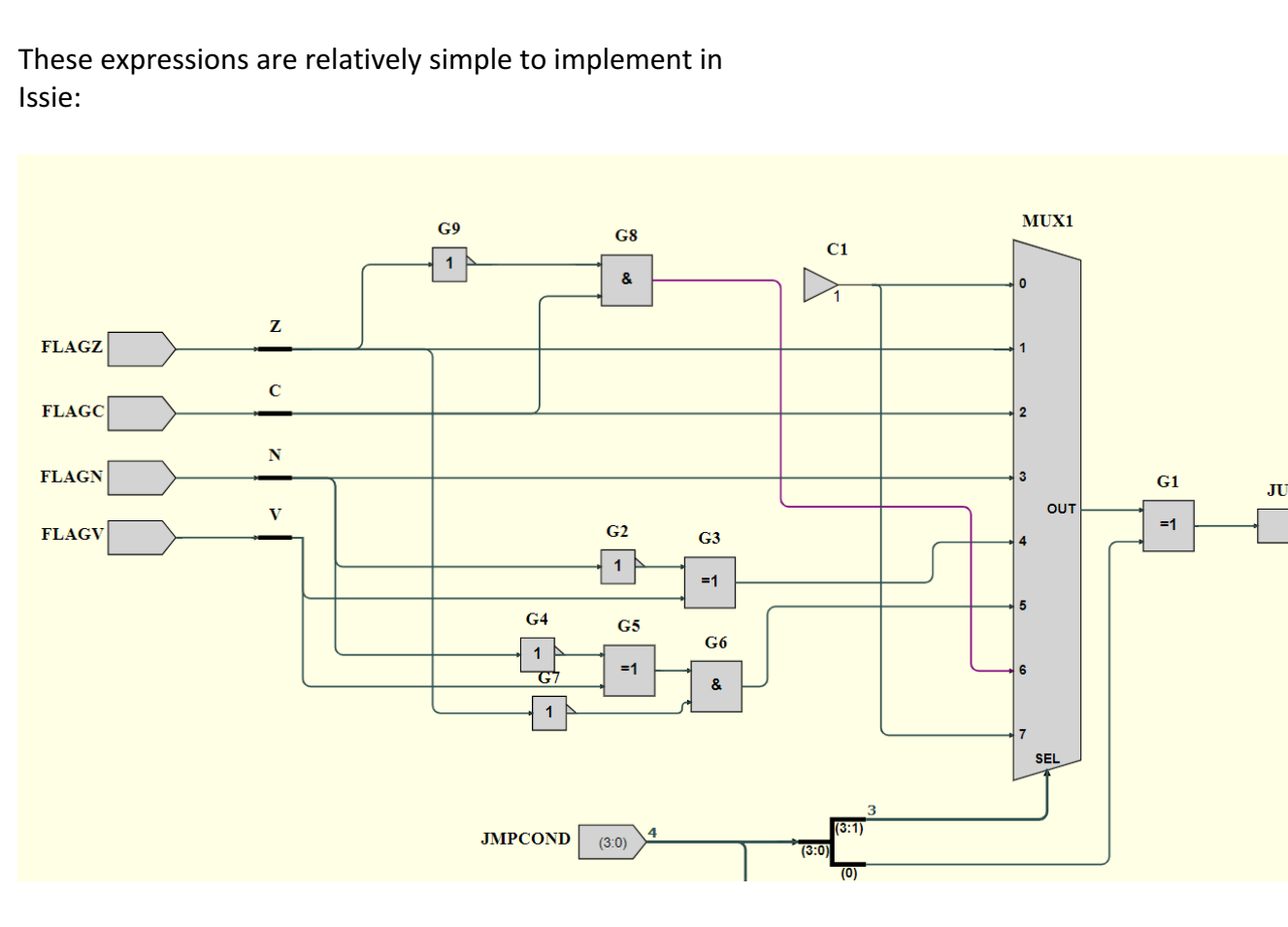
[4] Hardware is simpler because **JSR/RET** write/read R7 since **Ra** is normally read and written and for these instructions **a** = **JMPOPC(3:1)** = 7

Jump Instructions	Assembly	Operation	Examples
All except RET, JSR	JEQ JOFFSET; JEQ SYMBOL	PC := PC + JOFFSET if condition is true	`JMP -1 ; JGT LOOP ; JSR SUB1`
RET	RET	PC := R7	`RET`
JSR	JSR JOFFSET	R7 := PC+1; PC := PC + JOFFSET	`JSR 10`

Figure 2 - EEP1 JUMP instructions, from labs GitHub, github.com/edstott

- JMPOPC (3:1) = 4 : $\neg N \oplus V$
- JMPOPC (3:1) = 5 : $(\neg N \oplus V)\cdot\neg Z$
- JMPOPC (3:1) = 6 : $C\cdot\neg Z$

These expressions are relatively simple to implement in Issie:

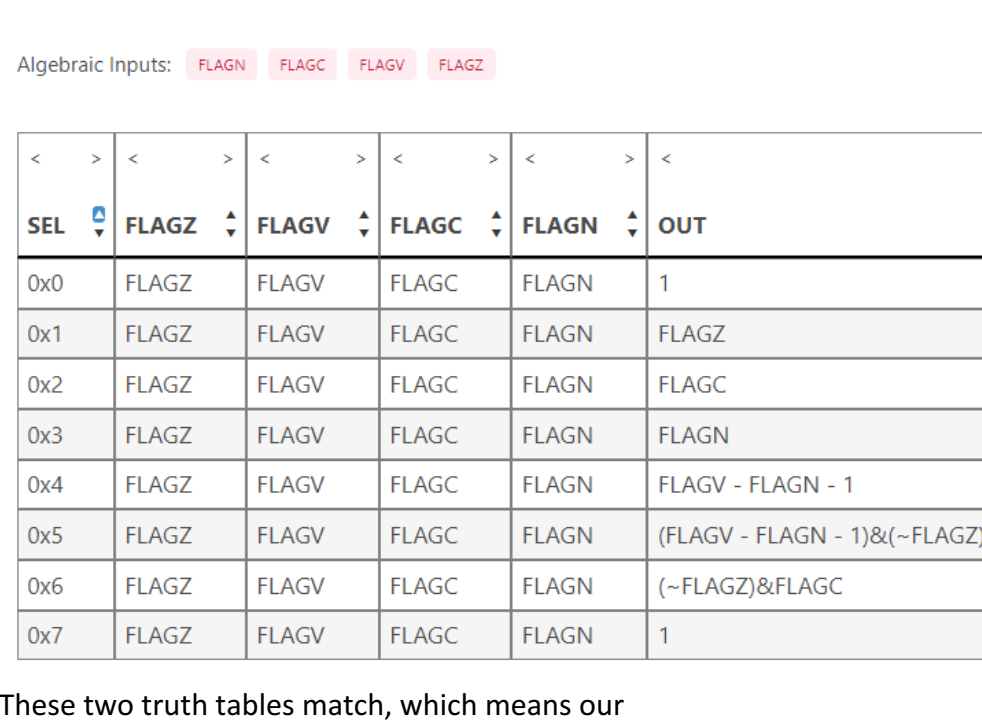


For JMPOPC (3:1) = 0 or 7, the output value can be set to constant 1, because for these JUMP instructions, there are no conditions - they always jump.
As for the cases in which JMPOPC (3:1) = 1, 2, or 3, the conditions are simply Z, C, and N respectively, so no further combinational logic needed there either.

This should theoretically be the correct version of the cond sheet. We can test this in Issie by making a truth table.
This is what we expect to see:

JMPOPC (3:1)	Flag Z	Flag C	Flag N	Flag V	Out
0	X	X	X	X	1
1	Z	X	X	X	Z
2	X	C	X	X	C
3	X	X	N	X	N
4	X	X	N	V	$\neg N \oplus V$
5	Z	X	N	V	$(\neg N \oplus V)\cdot\neg Z$
6	Z	C	X	X	$C\cdot\neg Z$
7	X	X	X	X	1

This is the algebraic truth table we get from Issie:



Algebraic Inputs: FLAGN FLAGC FLAGV FLAGZ

SEL	FLAGZ	FLAGV	FLAGC	FLAGN	OUT
0x0	FLAGZ	FLAGV	FLAGC	FLAGN	1
0x1	FLAGZ	FLAGV	FLAGC	FLAGN	FLAGZ
0x2	FLAGZ	FLAGV	FLAGC	FLAGN	FLAGC
0x3	FLAGZ	FLAGV	FLAGC	FLAGN	FLAGN
0x4	FLAGZ	FLAGV	FLAGC	FLAGN	FLAGV - FLAGN - 1
0x5	FLAGZ	FLAGV	FLAGC	FLAGN	(FLAGV - FLAGN - 1)&(~FLAGZ)
0x6	FLAGZ	FLAGV	FLAGC	FLAGN	(~FLAGZ)&FLAGC
0x7	FLAGZ	FLAGV	FLAGC	FLAGN	1

These two truth tables match, which means our combinational logic has been implemented correctly.

Remark: $V \oplus \neg N = V - N - 1$ for 1 bit numbers like we have here.

Task 3 - *Walk through the code in Figure 1 by hand with op1=12, op2 = 5 to check that it works correctly and generates 60 as the answer. Compare its operation with the shift and add multiply method explained in Lecture 3 to show how it works.*

```
// implement sum := op1 * op2 (LS 16 bits of)
// assume int = 16 bits (16 bit version of C).
unsigned int op1, op2, op2_shifted, sum; // variables
sum = 0; // initisalise
op2_shifted = op2; //initialise
while (op1 != 0) {
    if (op1 & 1) { // & bitwise AND operation
        sum = sum + op2_shifted;
    }
    op2shifted = op2shifted << 1; // left shift by 1
    op1 = op1 >> 1; // right shift by 1.
}
```

Figure 1 - while loop implementation of multiply, from labs GitHub, github.com/edstott

We want to implement 12 x 5 (op1 = 12 and op2 = 5).
The first step is to walk through the code by hand:

op1 != 0	op1 odd	op1	op2	op2 _shifted	sum
true	false	12	5	5	0
true	false	6	5	10	0
true	true	3	5	20	0
true	true	1	5	40	20
true	X	0	5	80	60

By the end of the program, we have sum equal to 60, which is indeed what we wanted, since we were multiplying 12 and 5.

Question: How does the bitwise AND operator (&) work in C?
Answer: In this case, we are bitwise ANDing the 16 bit number op1 and the number 1. So, in effect, we are doing the following:

	XXXX XXXX XXXX XXXz	= op1 (but we are only interested in the least significant bit)
&	0000 0000 0000 0001	$= (1)_2$
=	0000 0000 0000 000z	= true if z

Interpretation: & statement return true if op1 is an odd number, and false otherwise.

Let's now compare our method with the one from Lecture 3:
The operation we want to execute is 12 times 5.

$(12)_{10} = (1100)_2\,,\ (5)_{10} = (0101)_2$

$\mathrm{A} = 1100, \qquad \mathrm{B} = 0101$

$$\text{product} = \mathrm{A_3}\cdot\mathrm{B}\cdot 8 + \mathrm{A_2}\cdot\mathrm{B}\cdot 4 + \mathrm{A_1}\cdot\mathrm{B}\cdot 2 + \mathrm{A_0}\cdot\mathrm{B}\cdot 1 = \sum_{n=0}^{3}\mathrm{A_n}\cdot\mathrm{B}\cdot 2^{n}$$

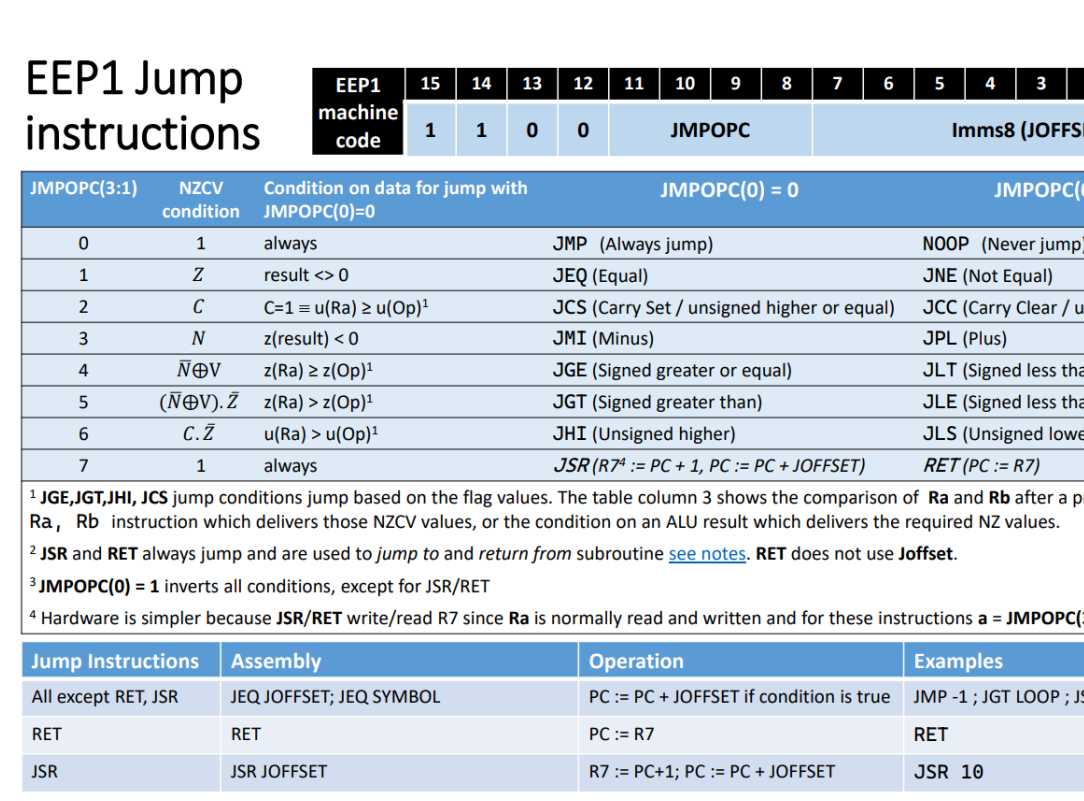
$$= 0101000 + 0010100 + 0000000 + 0000000 = 111100 = 60$$

This was indeed the answer we were expecting to get, which means we have been succesful. Task 3 is thus complete.

Task 4 - *Using the method from Lecture 4 for if-then and while constructs, and the EEP1 instructions for addition, shifting, logical AND, translate the program in Figure 1 into assembly language, using registers R0, R1, R2, R3 for the 4 required variables. Write your program in a mul1.txt file and use eepassembler to turn this into a mul1.ram file containing machine code. Run the file using the model EEP1 (or your own correctly working lab1 version) and check that it correctly multiplies.*

Our task is to translate the program from Figure 1, below, into assembly language, using only register 0 through 3.

```
// implement sum := op1 * op2 (LS 16 bits of)
// assume int = 16 bits (16 bit version of C).
unsigned int op1, op2, op2_shifted, sum; // variables
sum = 0; // initisalise
op2_shifted = op2; //initialise
while (op1 != 0) {
    if (op1 & 1) { // & bitwise AND operation
        sum = sum + op2_shifted;
    }
    op2shifted = op2shifted << 1; // left shift by 1
    op1 = op1 >> 1; // right shift by 1.
}
```



EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							

JMPOPC(3:1)	NZCV condition	Condition on data for jump with JMPOPC(0)=0	JMPOPC(0) = 0	JMPOPC(0) = 1[3]
0	1	always	**JMP** (Always jump)	**NOOP** (Never jump)
1	Z	result <> 0	**JEQ** (Equal)	**JNE** (Not Equal)
2	C	C=1 = u(Ra) ≥ u(Op)[1]	**JCS** (Carry Set / unsigned higher or equal)	**JCC** (Carry Clear / unsigned lower)
3	N	z(result) < 0	**JMI** (Minus)	**JPL** (Plus)
4	$\bar{N}\oplus V$	z(Ra) ≥ z(Op)[1]	**JGE** (Signed greater or equal)	**JLT** (Signed less than)
5	$(\bar{N}\oplus V).\bar{Z}$	z(Ra) > z(Op)[1]	**JGT** (Signed greater than)	**JLE** (Signed less than or equal)
6	$C.\bar{Z}$	u(Ra) > u(Op)[1]	**JHI** (Unsigned higher)	**JLS** (Unsigned lower or same)
7	1	always	***JSR*** *(R7[4] := PC + 1, PC := PC + JOFFSET)*	***RET*** *(PC := R7)*

[1] **JGE,JGT,JHI, JCS** jump conditions jump based on the flag values. The table column 3 shows the comparison of **Ra** and **Rb** after a previous SUB `Ra, Rb` instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.

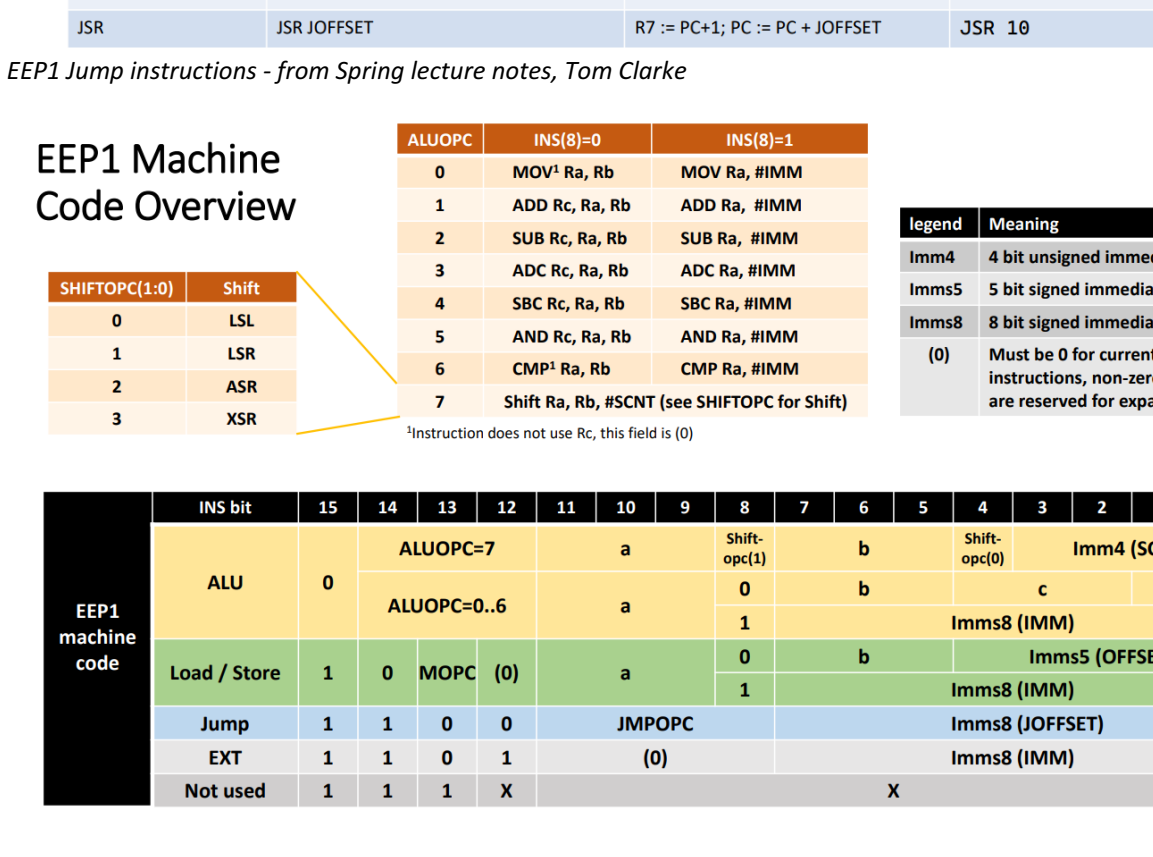
[2] **JSR** and **RET** always jump and are used to *jump to* and *return from* subroutine see notes. **RET** does not use **Joffset**.

[3] **JMPOPC(0) = 1** inverts all conditions, except for JSR/RET

[4] Hardware is simpler because **JSR/RET** write/read R7 since **Ra** is normally read and written and for these instructions **a** = **JMPOPC(3:1)** = 7

Jump Instructions	Assembly	Operation	Examples
All except RET, JSR	JEQ JOFFSET; JEQ SYMBOL	PC := PC + JOFFSET if condition is true	JMP -1 ; JGT LOOP ; JSR SUB1
RET	RET	PC := R7	RET
JSR	JSR JOFFSET	R7 := PC+1; PC := PC + JOFFSET	`JSR 10`

EEP1 Jump instructions - from Spring lecture notes, Tom Clarke



EEP1 Machine Code Overview

ALUOPC	INS(8)=0	INS(8)=1
0	MOV[1] Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP[1] Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

[1] Instruction does not use Rc, this field is (0)

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

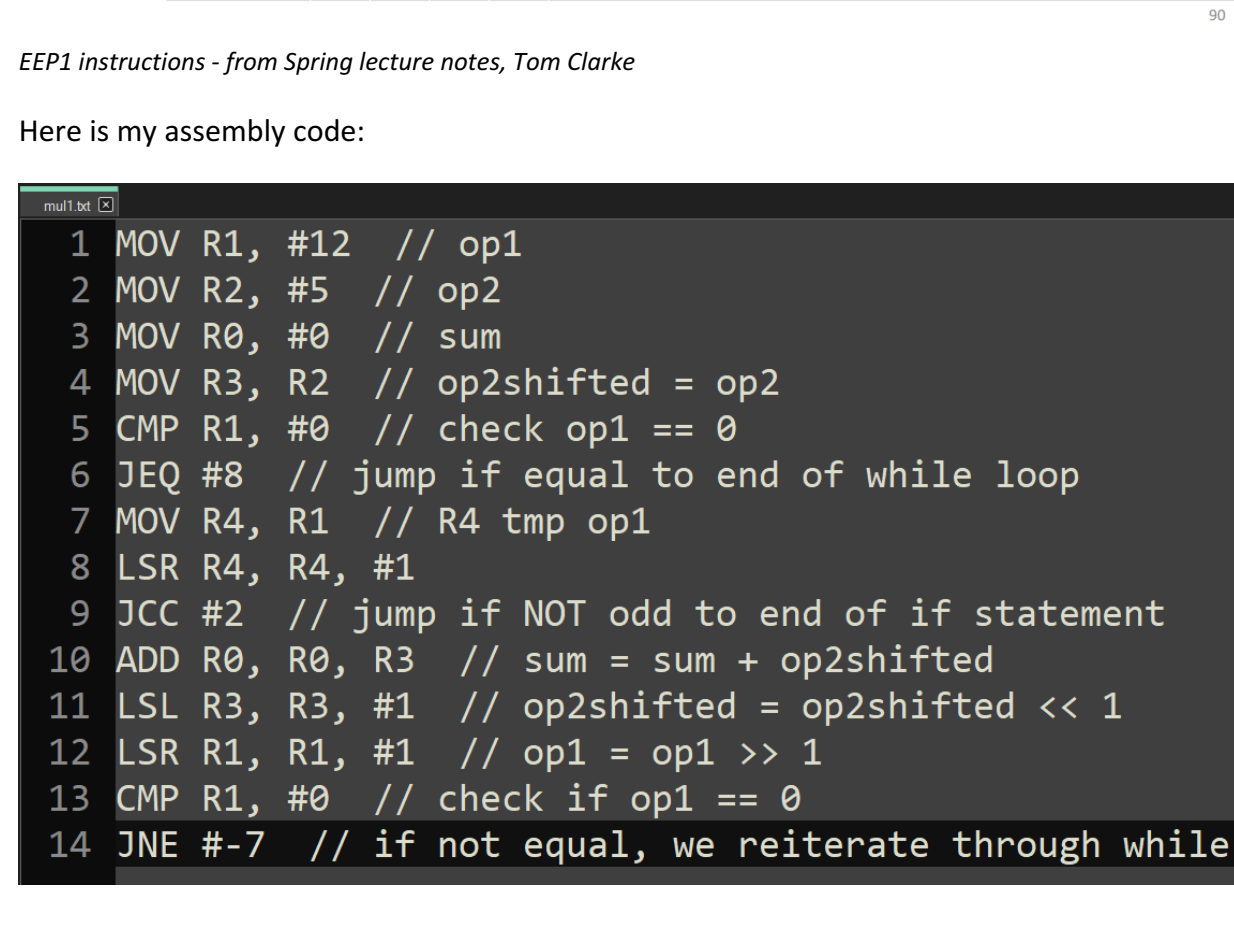
legend	Meaning
Imm4	4 bit unsigned immediate
Imms5	5 bit signed immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EEP1 machine code	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							
	Load / Store	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
									1	Imms8 (IMM)							
	Jump	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							
	EXT	1	1	0	1	(0)				Imms8 (IMM)							
	Not used	1	1	1	X	X											

90

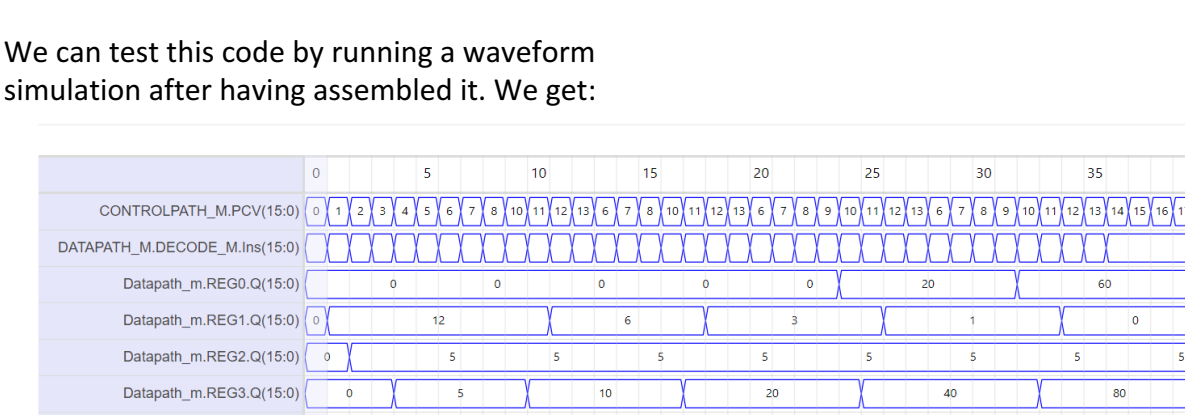
EEP1 instructions - from Spring lecture notes, Tom Clarke

Here is my assembly code:

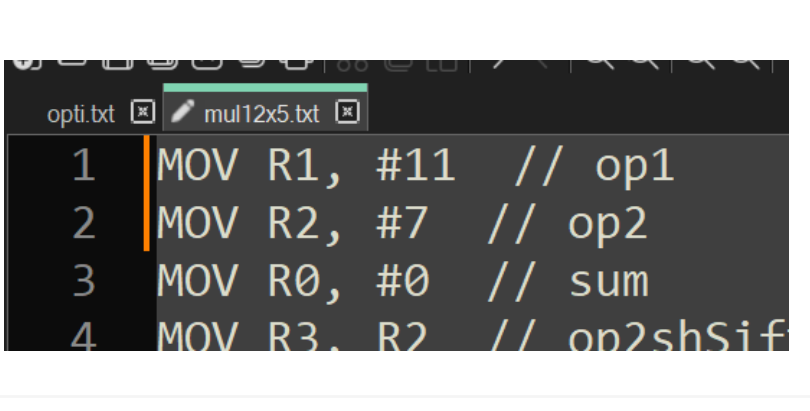


```
1 MOV R1, #12  // op1
2 MOV R2, #5  // op2
3 MOV R0, #0  // sum
4 MOV R3, R2  // op2shifted = op2
5 CMP R1, #0  // check op1 == 0
6 JEQ #8  // jump if equal to end of while loop
7 MOV R4, R1  // R4 tmp op1
8 LSR R4, R4, #1
9 JCC #2  // jump if NOT odd to end of if statement
10 ADD R0, R0, R3  // sum = sum + op2shifted
11 LSL R3, R3, #1  // op2shifted = op2shifted << 1
12 LSR R1, R1, #1  // op1 = op1 >> 1
13 CMP R1, #0  // check if op1 == 0
14 JNE #-7  // if not equal, we reiterate through while loop
```

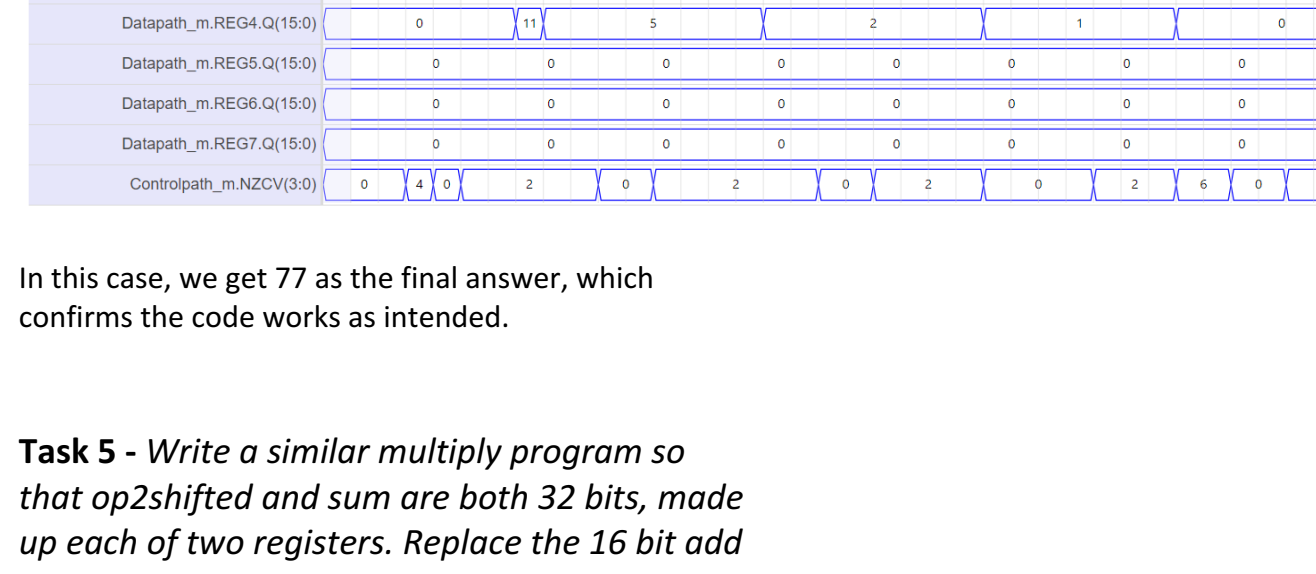
We can test this code by running a waveform simulation after having assembled it. We get:



Which is the correct answer: 12 x 5 is indeed 60.
Success!
We can try this with different numbers, for example 7 x 11, with only limited modification to the assembly code.



```
MOV R1, #11   // op1
MOV R2, #7   // op2
MOV R0, #0   // sum
MOV R3, R2   // op2shSif
```



In this case, we get 77 as the final answer, which confirms the code works as intended.

Task 5 - *Write a similar multiply program so that op2shifted and sum are both 32 bits, made up each of two registers. Replace the 16 bit add and shift operations by a corresponding 32 bit add and shift.*

My intuition is that we store a 32-bit number in a pair of 16-bit registers, such that one register store the least significant 16-bits of the 32-bit number, and the other stores the most significant 16-bits. We would also need to replace our LSR instruction by a XSR instruction, as extended shift right can perform shift operations on two registers at once, using Flag C.

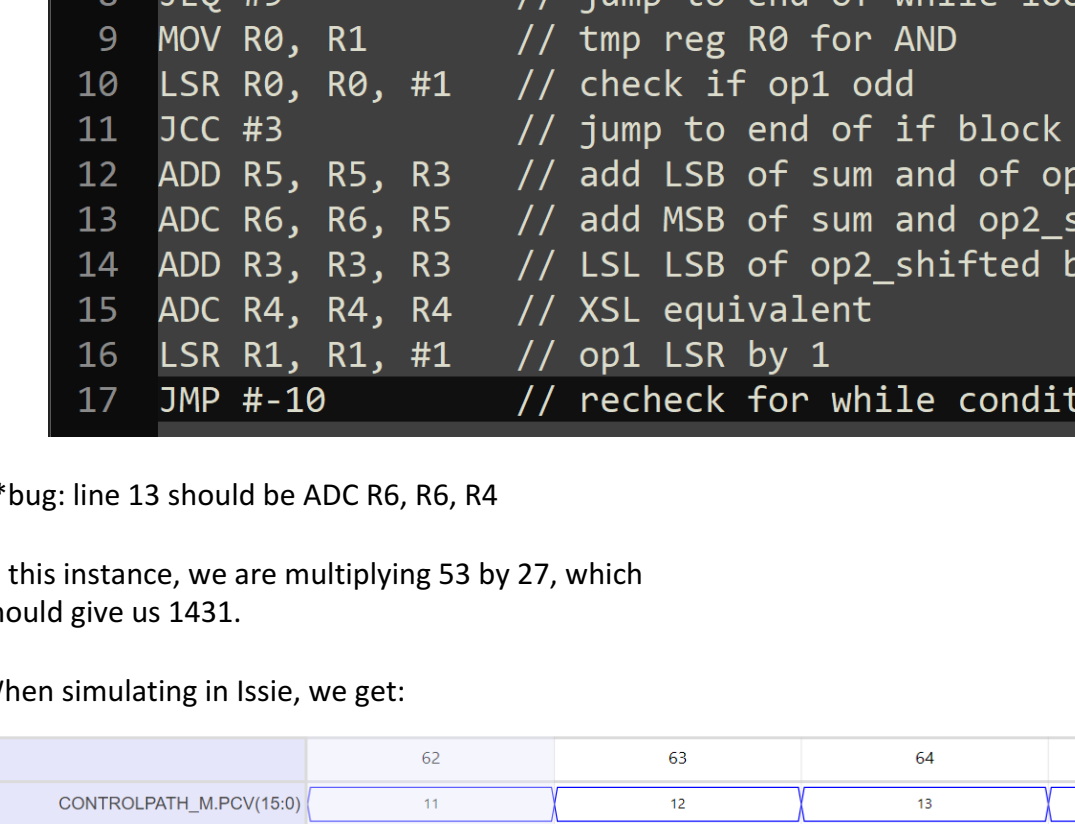
Correction: I was right about storing the 32-bit number in a pair of 16-bit registers. However, we don't have any use for XSR. We do, however, have a use for an XSL instruction, which does not exist. Fortunately, there is a rather simple workaround. We replace the XSR instruction by an ADD instruction which adds the least significant bits of a number (multiplying by 2 is equivalent to shifting left by 1), followed by an ADC instruction, which does the same thing for the most significant bits, but this time taking into account the carry from the previous ADD.

Example: if op2_shifted is a 32 bit number, stored such that its least significant 16 bits are in R1 and its *most* significant 16 bits are in R2, then XSL can be implemented as follows:

ADD R1, R1, R1
ADC R2, R2, R2

We have thus successfully modified a 32-bit number by 2.

Here is my assembly code:

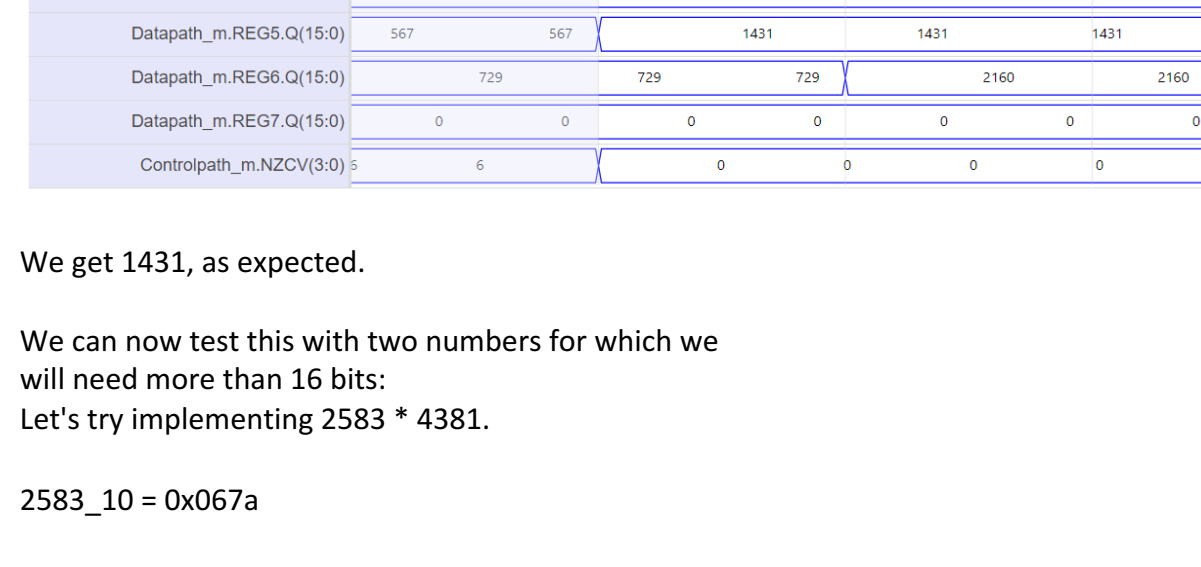


```
1  MOV R1, #53      // op1 = 53
2  MOV R2, #27      // op2 = 27
3  MOV R5, #0       // LSB of sum = 0
4  MOV R6, #0       // MSB of sum = 0
5  MOV R3, R2       // LSB of op2_shifted = op2
6  MOV R4, #0       // MSB of op2_shifted = 0
7  CMP R1, #0       // check op1 == 0
8  JEQ #9           // jump to end of while loop
9  MOV R0, R1       // tmp reg R0 for AND
10 LSR R0, R0, #1   // check if op1 odd
11 JCC #3           // jump to end of if block
12 ADD R5, R5, R3   // add LSB of sum and of op2_shifted
13 ADC R6, R6, R5   // add MSB of sum and op2_shifted, wi
14 ADD R3, R3, R3   // LSL LSB of op2_shifted by 1
15 ADC R4, R4, R4   // XSL equivalent
16 LSR R1, R1, #1   // op1 LSR by 1
17 JMP #-10         // recheck for while condition
```

**bug: line 13 should be ADC R6, R6, R4

In this instance, we are multiplying 53 by 27, which should give us 1431.

When simulating in Issie, we get:



We get 1431, as expected.

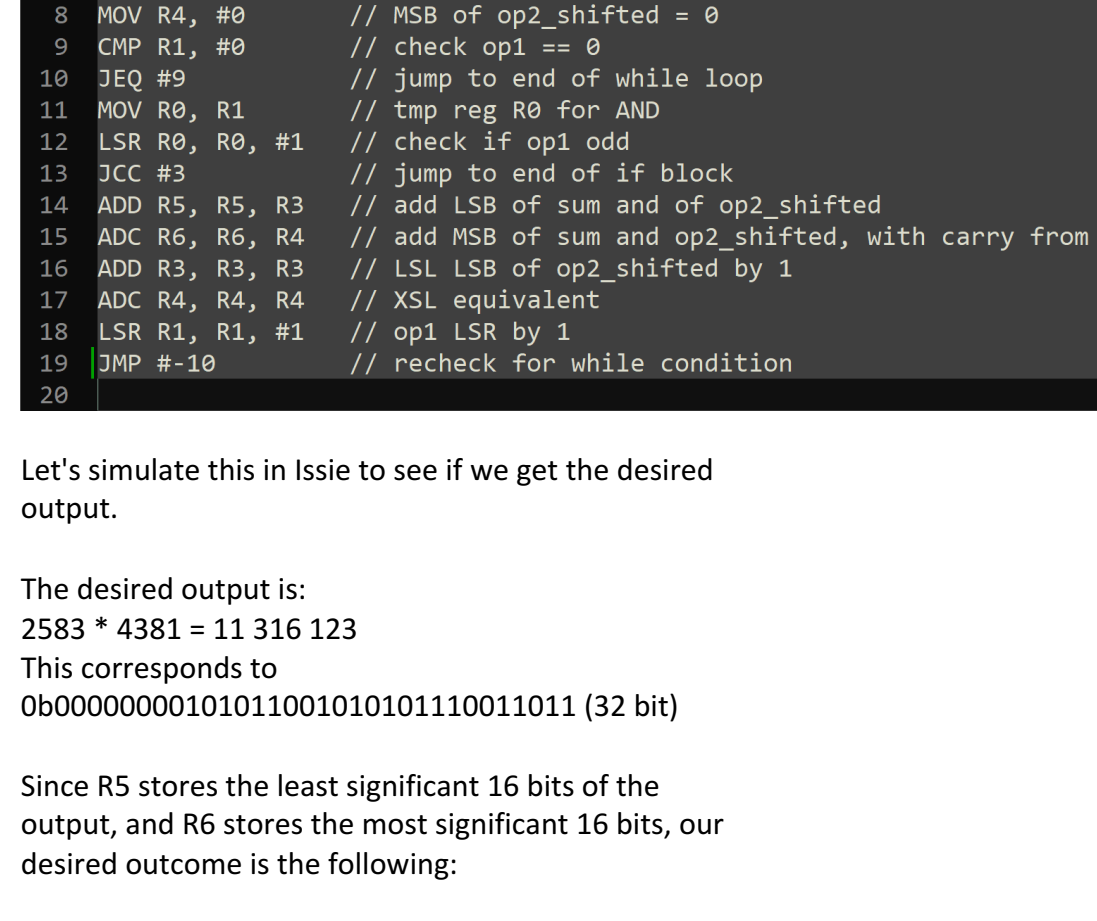
We can now test this with two numbers for which we will need more than 16 bits:
Let's try implementing 2583 * 4381.

2583_10 = 0x067a

4381_10 = 0x111d

In order to use MOV with immediate values which are greater than the maximum value for an 8 bit signed integer (127), we use the EXT instruction.

Here is the code implementing this:

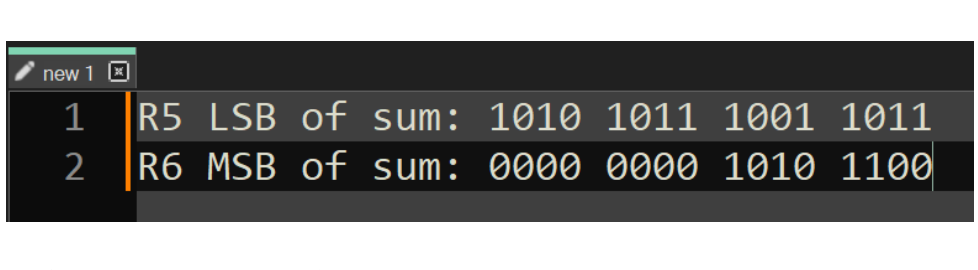


```
EXT 0x0A
MOV R1, #0x17     // op1 = 53
EXT 0x11
MOV R2, #0x1D      // op2 = 27
MOV R5, #0        // LSB of sum = 0
MOV R6, #0        // MSB of sum = 0
MOV R3, R2        // LSB of op2_shifted = op2
MOV R4, #0        // MSB of op2_shifted = 0
CMP R1, #0        // check op1 == 0
JEQ #9            // jump to end of while loop
MOV R0, R1        // tmp reg R0 for AND
LSR R0, R0, #1    // check if op1 odd
JCC #3            // jump to end of if block
ADD R5, R5, R3    // add LSB of sum and of op2_shifted
ADC R6, R6, R4    // add MSB of sum and op2_shifted, with carry from previous add
ADD R3, R3, R3    // LSL LSB of op2_shifted by 1
ADC R4, R4, R4    // XSL equivalent
LSR R1, R1, #1    // op1 LSR by 1
JMP #-10          // recheck for while condition
```

Let's simulate this in Issie to see if we get the desired output.

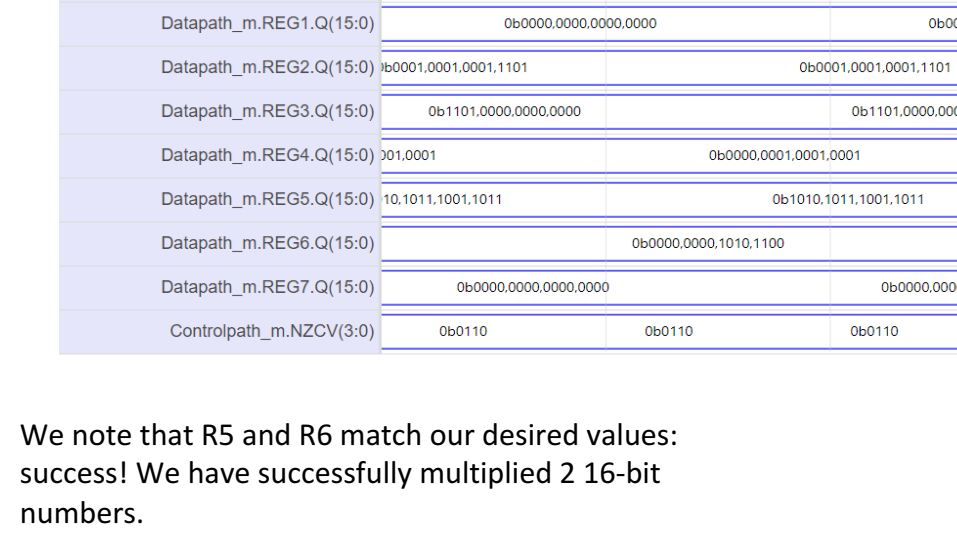
The desired output is:
2583 * 4381 = 11 316 123
This corresponds to
0b00000000101011001010101110011011 (32 bit)

Since R5 stores the least significant 16 bits of the output, and R6 stores the most significant 16 bits, our desired outcome is the following:



```
R5 LSB of sum: 1010 1011 1001 1011
R6 MSB of sum: 0000 0000 1010 1100
```

Let's now run the waveform simulation in Issie to check if we do indeed get the desired output:

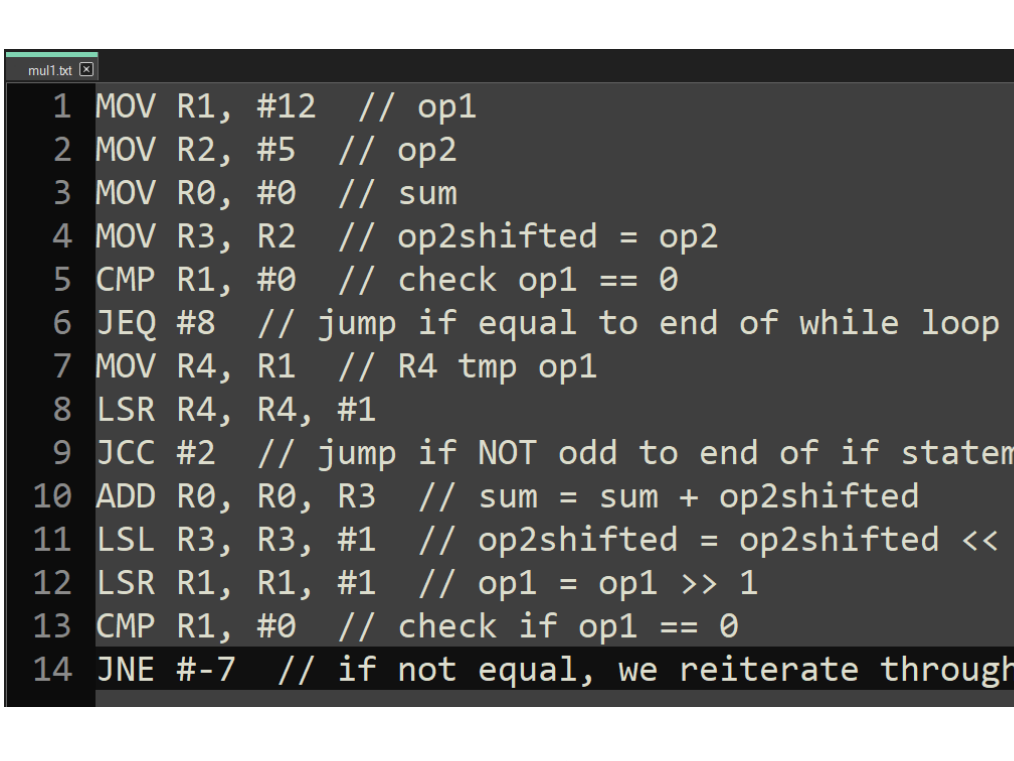


We note that R5 and R6 match our desired values: success! We have successfully multiplied 2 16-bit numbers.

Challenges

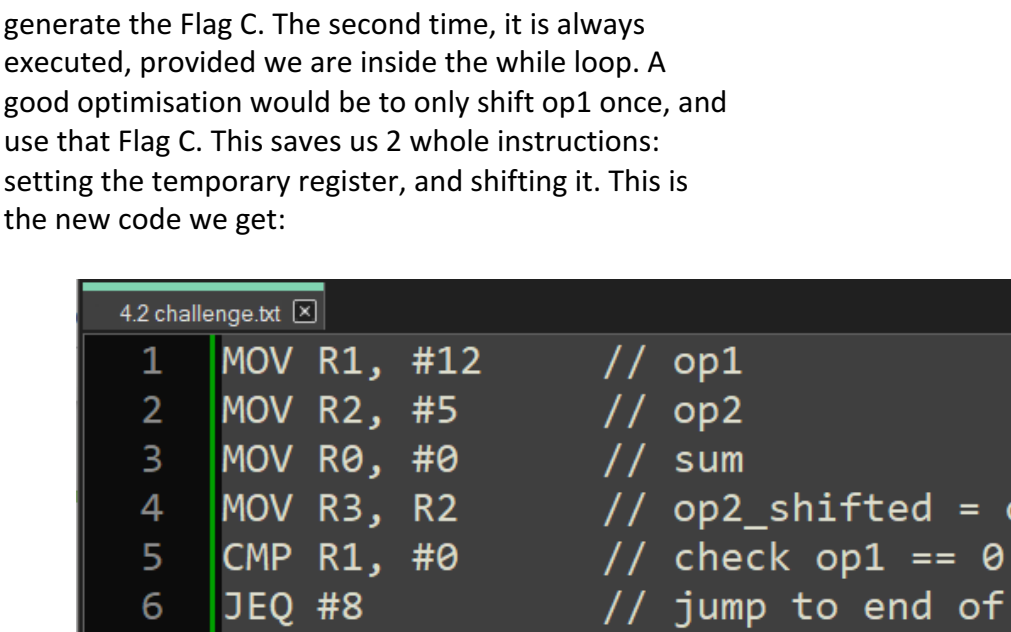
5.1 - Software - Optimising code contained in the while loop

The first step in the software challenges is to optimise the code within the while loop. For reference, here is the original code:



```
1 MOV R1, #12  // op1
2 MOV R2, #5  // op2
3 MOV R0, #0  // sum
4 MOV R3, R2  // op2shifted = op2
5 CMP R1, #0  // check op1 == 0
6 JEQ #8  // jump if equal to end of while loop
7 MOV R4, R1  // R4 tmp op1
8 LSR R4, R4, #1
9 JCC #2  // jump if NOT odd to end of if statement
10 ADD R0, R0, R3  // sum = sum + op2shifted
11 LSL R3, R3, #1  // op2shifted = op2shifted << 1
12 LSR R1, R1, #1  // op1 = op1 >> 1
13 CMP R1, #0  // check if op1 == 0
14 JNE #-7  // if not equal, we reiterate through while loop
```

We can notice we perform LSR on op1 twice: once on line 8, then again on line 12. The first time, it is to check if op1 is odd - as that is the condition for the if statement - we are only executing that instruction to generate the Flag C. The second time, it is always executed, provided we are inside the while loop. A good optimisation would be to only shift op1 once, and use that Flag C. This saves us 2 whole instructions: setting the temporary register, and shifting it. This is the new code we get:



```
MOV R1, #12      // op1
MOV R2, #5       // op2
MOV R0, #0       // sum
MOV R3, R2       // op2_shifted = op2
CMP R1, #0       // check op1 == 0
JEQ #8           // jump to end of while loop
LSR R1, R1, #1
JCC #2           // jump to end of if statement
ADD R0, R0, R3   // sum = sum + op2_shifted
LSL R3, R3, #1   // op2_shifted << 1
CMP R1, #0
JNE #-7
```

We can test the real world impact of this change. Previously, in order to multiply 12 and 5, EEP1 took 31 clock cycles. However, when simulating with the new code, the same multiplication also takes 31 clock cycles. It seems no gain has been made there, nevertheless we did free a register from being used: R4 is never used during the process.